

Do We Still Need to Be Agile? What Two Cohorts of Student Projects Reveal About Planning Discipline in the GenAI-Era

Abstract—Generative AI can make local software implementation faster, but it may also raise the importance of shared architectural intent, integration, and human coordination. This study examines that tension across two cohorts of student software projects comprising 14 team-semesters and three synchronized checkpoints. We combine multi-checkpoint surveys, repository mining, independent blind-scored evaluator scorecards, and transcript-based evidence to study Generative AI adoption and role perceptions, the temporal density of repository activity, and associations between upfront architectural artifacts, late rework, and project outcomes. The results show that self-reported AI use is widespread and task-oriented, but perception families do not form a single longitudinal trajectory. Repository activity and clean source/test change concentrate near final integration, while candidate coordination-language density increases across the observed 2025.2 transcript checkpoints. Observable T1 architectural-artifact scope varies substantially, yet its associations with T3 clean-rework magnitude, baseline-conditioned rework ratios, and independent outcomes are mixed and exploratory. These findings suggest that Agile remains necessary, but AI-assisted development requires a more explicit and inspectable layer of architectural planning, interfaces, dependencies, and acceptance conditions. Spec-Driven Development is discussed as a plausible, testable response to this coordination pressure, not as an intervention validated by the present observational study.

I. INTRODUCTION

The Agile Manifesto revolutionized software engineering by prioritizing working software, individual interactions, and responding to change over comprehensive documentation and strict upfront planning. For over two decades, this empirical paradigm was the gold standard for navigating requirements uncertainty, and for good reason: in a world where typing and boilerplate were the bottlenecks, minimizing upfront documentation minimized waste.

That world is changing. Generative AI assistants now produce substantial amounts of code in seconds, shifting scarce software-engineering effort toward architectural alignment, integration, and cognitive coordination. Under this new cost equation, the Agile bet—that planning can remain deliberately light as understanding develops—deserves to be tested rather than assumed.

If deferring design in an AI-assisted environment merely relocates effort, the result may be a collision between expectations of rapid AI-enabled productivity and the realities of late-stage architectural integration. This paper examines that possibility through repository metadata, multi-checkpoint student surveys, independent evaluator scorecards, and qual-

itative meeting transcripts across two cohorts of software-engineering projects. The resulting evidence connects what students report, what repositories reveal, what evaluators score, and what coordination language appears in the project record.

We structure the investigation around three questions:

- **RQ1 (Descriptive):** How do student developers utilize Generative AI, and how does this adoption shape their longitudinal perceptions of software engineering roles and project expectations?
- **RQ2 (Descriptive):** How does the temporal density of repository activity contrast with the qualitative typification of human coordination friction across the project lifecycle?
- **RQ3 (Primary):** How does the quality of upfront architectural artifacts (evaluated at baseline T_1) associate with late-stage destructive rework and independent, blind-scored project outcomes?

The study makes three contributions. First, it provides a longitudinal, multi-method account of AI-assisted student development that keeps perceived AI impact, self-reported adoption, repository activity, and qualitative coordination evidence analytically distinct. Second, it makes the temporal contrast between repository activity and integration pressure observable while preserving the different grains and coverage of the underlying evidence. Third, it examines upfront architectural-artifact evidence, late clean-path rework, and independent outcomes without collapsing structural observations into an unsupported quality score. The resulting picture is deliberately more useful than a simple success/failure verdict: it shows where implementation speed meets the harder problem of shared system intent.

The findings motivate a practical proposition. Generative AI does not remove the need for Agile practice; it increases the importance of making architectural intent, interfaces, dependencies, and acceptance conditions explicit while remaining open to revision. Spec-Driven Development (SDD) is a promising way to organize that layer by carrying a specification through technical planning, task decomposition, implementation, and convergence checks [1], [2]. We present SDD as a plausible, testable response to the coordination pressure observed in AI-assisted work, not as an intervention evaluated by this study. The empirical findings and the proposition together invite a more precise question for future Agile practice: how can teams preserve adaptability while ensuring

that people and AI systems act on the same inspectable account of what the system is meant to become?

II. BACKGROUND AND RELATED WORK

A. The Agile value hierarchy and coordination risk

The Agile Manifesto prioritizes working software, interaction, and response to change over comprehensive documentation [3]. This emphasis remains valuable when requirements evolve, but it does not imply that shared architectural intent is unnecessary. Classic accounts of software coordination emphasize that communication costs grow with system size and team interaction, especially when structure remains tacit [4], [5]. The relevant question for AI-assisted development is therefore not whether teams should return to immutable upfront design, but which forms of intent must remain explicit while implementation continues iteratively.

B. Generative AI in software engineering: From local generation to systems

Recent work presents Generative AI as a major change in software-engineering practice while emphasizing that its effects vary across tasks, users, and organizational contexts [6]–[8]. Empirical studies of code-generating tools show that programmers work through iterative interaction, verification, and contextual steering rather than simply accepting generated code [9]. This distinction is central to the present study: local code generation may accelerate an activity without resolving cross-file dependencies, architectural boundaries, or team coordination. Software-engineering education is already confronting this change, including questions about how students use and assess Generative AI in project work [10].

C. Upfront planning, architecture, and Spec-Driven Development

Architecture and technical-debt research treats design decisions as evolving constraints whose consequences may accumulate over time. Architectural debt research has developed concepts and tools for identifying architectural debt, following its accumulation, and relating it to later remediation effort [11], [12]. These traditions motivate measuring early architectural artifacts separately from semantic quality: an observed artifact is evidence of an observable practice, not proof that its design was adequate.

Spec-Driven Development (SDD) extends this concern into AI-assisted workflows. Current practitioner descriptions frame SDD as carrying a specification through planning, task decomposition, implementation, and convergence checks [1], [2]. In this paper, SDD is treated as a testable process proposition: making intent and constraints inspectable may reduce ambiguity when AI tools generate implementation across multiple files and contributors. Existing SDD guidance is used to define the proposition, not as evidence that SDD improves outcomes.

D. Technical debt, rework, and empirical measurement

Technical debt describes the future cost of expedient decisions, but its measurement is difficult because productive iteration can resemble rework in volume-based repository data [13]–[15]. Architecture-debt studies likewise distinguish observable signals from claims about design quality or defect causation. The present work follows that measurement discipline by separating activity volume, clean-path provenance, baseline eligibility, and evaluator outcomes rather than collapsing them into a single score.

E. Research gap and contribution

The related literature provides strong accounts of Agile coordination, GenAI adoption and interaction, architectural debt, and evolving software practice. What remains less visible is how these perspectives meet in student projects that use Generative AI while being observed across synchronized checkpoints. This study addresses that gap through a bounded mixed-method design combining survey reports, repository activity and path provenance, independent evaluator scorecards, and transcript-based coordination evidence. Its contribution is not a causal estimate of AI productivity or a validation of SDD. It is an evidence-preserving account of how local AI-assisted implementation, temporal integration pressure, explicit architectural artifacts, rework proxies, and independent outcomes can be examined together without silently merging their analytical grains.

III. METHODOLOGY

To investigate how planning discipline and coordination evolve in AI-assisted software engineering projects, we designed an empirical study combining longitudinal repository mining, standardized instructor evaluations, multi-checkpoint student surveys, and qualitative meeting transcripts across two academic offerings.

A. Context, Course Design, and “Rules of the Game”

The study was conducted within a project-based undergraduate software engineering course at the State University of Ceará (UECE), spanning two semesters: 2025.2 ($N = 9$ analyzable teams) and 2026.1 ($N = 5$ teams), yielding a total inferential dataset of $N = 14$ team-semesters (≈ 70 participating students).

Students formed teams of approximately five members through spontaneous self-allocation, with remaining students assigned to vacant slots to ensure balanced team sizes. Teams enjoyed complete autonomy regarding their internal organization, process models, and project management methodologies (e.g., choosing freely between Agile variants, Scrum, Kanban, or unguided approaches). However, course guidelines enforced two structural constraints:

- 1) Teams were required to develop an operational software project featuring an integrated Generative AI component as a core functional requirement. Specifically, projects

were expected to implement advanced architectural patterns such as Retrieval-Augmented Generation (RAG)—a technique that dynamically grounds Large Language Models (LLMs) in external knowledge bases to mitigate hallucination [16]—or LLM-driven data analytics pipelines.

- 2) Students were strongly encouraged to utilize generative AI assistants throughout all phases of software development, from requirements specification to implementation and testing.

The project lifecycle was anchored by three synchronized temporal checkpoints (T_1 , T_2 , and T_3) paced identically across both cohorts:

- T_1 (**Baseline**): Administered on the first day of class prior to any formal instruction on generative AI processes, capturing baseline perceptions, initial project scope, and early planning commits.
- T_2 (**Mid-Semester**): Conducted following structured instructional modules covering generative AI mechanics, RAG architectures, and software process models, accompanied by the second wave of survey instruments and status presentations.
- T_3 (**Final Integration**): Positioned one week prior to the final project delivery and presentation, capturing the peak pressure of system integration and deadline convergence.

B. Project Scope and Real-World Complexity

To ensure ecological validity and induce realistic coordination friction, the course guidelines explicitly prohibited trivial algorithmic exercises or “toy” applications. As part of the “rules of the game,” teams were mandated to identify and solve real-world community, educational, or business problems. This constraint forced students to grapple with high domain complexity, authentic user requirements, and the nontrivial challenge of embedding Generative AI into functional architectures.

The resulting software systems spanned diverse domains, ranging from healthcare and financial management to academic research and ecotourism. Table I details the 14 projects analyzed in this study. The breadth and complexity of these applications validate that the observed late-stage integration challenges and chronic coordination friction were driven by authentic software engineering workloads rather than artificially simplistic academic tasks.

C. Data Sources and Collection Framework

Our data collection framework triangulates four distinct evidentiary channels to capture both technical artifacts and human behavioral dynamics.

1. Git Repositories: Complete historical commit logs, branch structures, line change counts, and file-level provenance were extracted from all team repositories across the entire semester lifecycle. Students were explicitly instructed to centralize all project artifacts—including requirements, architectural specifications, documentation, and source code—directly within their Git repositories. This directive defined

the repository as the intended source of truth for the team’s production effort. The analysis used read-only repository histories and standardized event tables; Git remains repository evidence, not AI-use telemetry.

2. Independent Evaluator Scorecards: At each checkpoint (T_1 , T_2 , T_3), teams delivered formal status presentations. These were assessed by a panel of independent supervisors (4 evaluators in the 2025.2 cohort, and 3 in the 2026.1 cohort). To prevent groupthink and ensure scoring validity, an independent blind-scoring protocol was enforced: following the presentation and open feedback, each supervisor submitted their scores via a private electronic form. Evaluators remained blind to their peers’ scores throughout the semester. The final project score was calculated as an unweighted average of these independent assessments. Projects were assessed across four structured criteria on explicit bounded scales: *Project Evolution* (0 to 4.0 points), *Scope and Applicability* (0 to 2.0 points), *Technical Complexity* (0 to 2.0 points), and *Engagement and Participation* (0 to 2.0 points). These observations were organized by team-semester and checkpoint for analysis.

3. Multi-Checkpoint Student Surveys: Students completed identical survey instruments at T_1 , T_2 , and T_3 (totaling six instrument cycles). These surveys systematically captured data across five core constructs: Demographics, Professional Experience, GenAI Adoption, Perceptions of AI Impact on SE Roles, and Project Expectations. To ensure replicability, the core dimensions and sample variables tracked across these checkpoints are detailed in Table II. Survey responses are cohort-by-semester/checkpoint data and carry no team identifier; M1 and M2 therefore remain at that grain.

4. Qualitative Meeting Transcripts: To capture coordination dynamics and evaluative feedback, audio recordings were collected from the three formal status presentation checkpoints. This dataset comprises the full audio records of the student presentations alongside the subsequent deliberative discussions among the supervisors in a private chat group. Crucially, these supervisory discussions were free-form qualitative reflections, completely decoupled from the numeric scorecasting process. To make the transcription data computationally feasible for analysis, the raw text was segmented into analytical chunks, yielding 100 distinct fragments for the 2025.2 cohort (35 at T_1 , 37 at T_2 , and 28 at T_3). M5 reconstructs one global corpus per checkpoint; chunks and source sessions are provenance units, not independent observations. Transcript coverage for 2026.1 is unavailable and is never imputed.

Data Anonymization and Privacy: To ensure strict ethical compliance and protect participant confidentiality, all raw data streams were subjected to a rigorous anonymization pipeline, combining deterministic heuristics with LLM-assisted redaction to scrub unstructured text across commit messages, survey free-text responses, and audio transcripts. Finalized analytical artifacts exclude direct personal identifiers. Detailed call records and raw payload provenance remain restricted to the internal reproducibility record.

TABLE I
ANALYZED STUDENT PROJECTS BY COHORT, DEMONSTRATING REAL-WORLD PROBLEM SCOPE AND GENERATIVE AI INTEGRATION

Semester	Project Name	System Description & AI Integration Scope
2026.1	FoodGuard	Web platform utilizing nutritional data and medical history to identify food allergens and prevent adverse reactions.
2026.1	RevisAI	Educational system that generates intelligent flashcards and organizes customized weekly study review cycles.
2026.1	SmartFlowIA	SaaS WhatsApp chatbot focusing on intelligent onboarding, orientation, and welcoming for university freshmen.
2026.1	Interview Prep IA	AI-driven system that analyzes resumes against job vacancies, simulates interviews, and provides personalized candidate feedback.
2026.1	EnemBot	Domain-specific chatbot designed to tutor and assist students with questions regarding the Brazilian National High School Exam.
2025.2	UECEEncontra	Intelligent virtual assistant and campus navigation system designed to help students and professors locate university resources.
2025.2	EstudaAI	Study management platform that processes multimodal files (audio, image, PDF) to generate lecture summaries, reminders, and track grades.
2025.2	CareerPath IA	Resume analysis tool that uses GenAI to match user profiles with market demands and suggest targeted skill development.
2025.2	Buliçoso	Healthcare application that utilizes LLMs to simplify complex medical leaflets and manage medication reminders for the elderly.
2025.2	EcoRoteiro	Ecotourism platform that generates personalized travel itineraries based on user profiles, physical conditions, and specific interests.
2025.2	Research Flow	Academic research assistant that searches articles, generates intelligent summaries, and formats texts to academic standards (e.g., ABNT, APA).
2025.2	ExamForge	Assessment simulator that ingests past exams and course materials to generate new tests matching specific professor grading patterns.
2025.2	SoFiA	Financial management web platform utilizing AI to help university students dynamically organize their personal finances.
2025.2	Synapse	Requirements engineering tool that summarizes technical client meetings to automatically generate functional requirements, user stories, and backlogs.

TABLE II
STRUCTURE OF THE MULTI-CHECKPOINT SURVEY INSTRUMENT

Construct	Evaluated Variables & Inquiries	Data Type
Demographics & Academic Background	Age, Gender, Current academic semester, Previous degrees, High school educational background.	Categorical
Professional SE Experience	Years of software development experience, Current employment status, Preferred engineering role (e.g., Backend, QA, ML), Largest past team size.	Categorical
Generative AI Adoption & Tooling	Frequency of AI usage, Specific tools adopted (e.g., ChatGPT, Gemini, GitHub Copilot, Claude), Primary use cases (e.g., Code generation, Debugging, Documentation).	Categorical & Multiple Choice
Perceived AI Impact on SE Roles	Perceived risk of extinction or severe disruption for specific roles: Backend, Frontend, QA, Project Manager, Product Manager, Scrum Master.	5-point Likert
Expectations & Motivation	Confidence levels prior to project initiation, Motivation to learn new techniques, Expected 5-year impact of AI on the industry.	5-point Likert & Open-ended

D. Statistical and Analytical Concepts

The analysis uses descriptive summaries to preserve the scale and grain of each evidence stream. Means are reported with standard deviations when the metric is an approximately interval-like score; medians and interquartile ranges are used for skewed counts and ratios. Proportions retain their denominators, and rolling seven-day windows are indexed relative to evaluator anchors; overlapping windows are therefore summaries of nearby periods, not independent observations.

For exploratory associations, Spearman’s rank correlation ρ measures the strength and direction of a monotonic relationship between two ranked variables without assuming a linear scale [17]. Its handling of ties is retained in the metric definition; study-specific limits from sample size, missing-

ness, and influential observations are addressed in Threats to Validity. M9 reports exploratory p-values as descriptive diagnostics rather than confirmatory significance tests. Leave-one-out analysis recomputes an association after removing each eligible team-semester in turn; the resulting range shows sensitivity to influential cases and is not a confidence interval. These summaries are always interpreted with their denominator, stratum, and measurement-availability state.

E. Advanced Metrics Operationalization

Sections IIIA–D established what was collected. This subsection formalizes how repository history, evaluator scorecards, survey responses, and transcript text become nine metric families used for RQ1–RQ3. Each family retains its analytical grain and exposes components only when they represent

distinct constructs; no component is silently combined into a new score.

A note on $T_1/T_2/T_3$ for repository-derived measures. The checkpoint labels for commit and file history denote windows bounded by each team’s evaluator presentation dates, not single calendar days. This distinction is retained throughout the repository-derived families and is made explicit in the corresponding artifacts and provenance comments.

1) *RQ1: How do student developers utilize Generative AI, and how does this adoption shape their longitudinal perceptions of software engineering roles and project expectations?*: RQ1 is operationalized at cohort-by-semester/checkpoint grain because the anonymized survey responses contain no team identifier. M1 separates the perception panel from self-reported usage frequency, tasks, tools, prior experience, autonomy/tool balance, and exploratory career topics. M2 preserves role-specific Likert distributions and paired sensitivity summaries. These families describe self-report and perceived role disruption; neither is AI-use telemetry or objective extinction risk.

M1 – Perception and self-reported usage panel. The six perception families remain separate. Descriptive distributions are reported alongside raw usage fields and missingness; no heterogeneous family average is interpreted as a behavioral dependency measure. Career-topic coding remains exploratory.

M2 – Perceived role disruption. Role-specific Likert responses are summarized by cohort, checkpoint, and role. Where privacy-preserving respondent identifiers permit it, paired T1–T3 sensitivity summaries are reported without exposing identities.

2) *RQ2: How does the temporal density of repository activity contrast with the qualitative typification of human coordination friction across the project lifecycle?*: **M3 – Author activity dynamics.** M3 separates phase activity, author concentration, and rolling seven-day activity. A secondary pooled relative-window view removes the semester dimension only for visualization; it does not replace semester-stratified observations.

M4 – Clean change dynamics. M4 separates clean-change magnitude, touching-commit intensity, artifact composition, and rolling clean churn. The consistent path-classification rule defines the included source and test paths; commit counts remain contextual rather than the primary M4 outcome.

M5 – Coordination evidence. M5 reconstructs one global transcript corpus per checkpoint and deterministically counts versioned lexical markers for alignment, handoff, integration, blockers, and rework. It reports density, composition, corpus coverage, and an auditable chunk queue. M5 is not joined or correlated with team-semester M3/M4 observations.

3) *RQ3: How does the quality of upfront architectural artifacts (evaluated at baseline T_1) associate with late-stage destructive rework and independent, blind-scored project outcomes?*: **M6 – Upfront architectural artifacts.** M6a (T1 architectural-artifact evidence) reports T1 artifact presence and repository-visible scope. Approved M6b (structured T1 planning-content evidence) reports separate categories—goals, architecture/design, task decomposition, risk/dependency, and

TABLE III
SUMMARY OF METRIC FAMILIES AND COMPONENTS

ID	Metric family and components	Primary grain
M1	Perception panel and self-reported usage components	Cohort × checkpoint
M2	Role-specific perceived disruption components	Cohort × checkpoint × role
M3	Activity, author concentration, and rolling activity	Team-semester / window
M4	Clean magnitude, intensity, composition, and trajectory	Team-semester / checkpoint / window
M5	Transcript density, composition, and coverage	Global corpus / checkpoint
M6	Upfront architectural artifacts and structured evidence	Team-semester
M7	Repository inactivity trajectory and checkpoint pattern	Team-semester / window
M8	Destructive rework proxy, ratio, trajectory, and eligibility	Team-semester / window
M9	Artifact–rework–outcome associations and influence diagnostics	Team-semester / stratum

literal evidence quotes—for nine observed team-semesters; five cases without T1 commit subjects were not measured. M6b is exploratory and is never reduced to a composite planning-quality score.

M7 – Repository inactivity. M7 reports recent seven-day repository inactivity around evaluator checkpoints and in T3-relative rolling windows. It describes repository timing and coverage only; it does not identify planning omission and is not an independent M9 predictor.

M8 – Destructive rework proxy with baseline. M8 separates clean rework magnitude, the clean rework ratio, the late-stage rolling trajectory, and baseline eligibility. This is a path-provenance proxy for destructive rework, not semantic defect validation. A path first observed at T1 or T2 is eligible for the provenance analysis; baseline-ineligible cases remain separate and are not imputed as stability or low quality.

M9 – Architectural artifacts, destructive rework, and independent outcomes. M9 reports descriptive Spearman associations between T1 architectural-artifact predictors and M8a/M8b or independent, blind-scored T3 evaluator outcomes, restricting ratio analyses to the baseline-eligible stratum. Approved M6b structured planning-content fields enter a separate non-composite association output for observed cases. Leave-one-out diagnostics expose influence. No M7 predictor or floor-imputation analysis is used.

Table III summarizes the nine metric families, their components, and their primary analytical grains.

TABLE IV
METRIC-BOUND LANGUAGE-MODEL TASK

Task	Feeds	Structured output
Planning evidence extraction	M6b (structured T1 planning content)	Goals, architecture/design, task decomposition, risk/dependency, and literal evidence quotes from T_1 commit subjects.

F. LLM Usage in Data Extraction, Processing, and Metrics

Language-model assistance is used selectively for data extraction and is kept separate from deterministic metric construction. M1 uses coded survey responses, and M5 uses deterministic lexical coding of the transcript corpus; neither metric receives a new language-model score. M6b uses structured extraction from T_1 commit subjects for nine observed team-semesters; five team-semesters have no T_1 subjects and remain unavailable rather than being imputed.

Table IV summarizes the metric-bound language-model task and the metric component it supports.

M6b outputs were reviewed by the research team and retained for exploratory analysis as separate categories. They are not reduced to a composite planning-quality or architectural-quality score.

Persistence and auditability. Model-assisted outputs are retained with their associated prompts, responses, hashes, usage metadata, and status so that each reported value can be traced to its source evidence. These records support reproducibility while remaining outside the paper-facing narrative.

Inference boundary. All reported metric families and components are descriptive or exploratory. Missing evidence remains distinct from zero or low quality, incompatible analytical grains are not silently combined, and no causal claim is made without a design that supports it.

IV. RESULTS

This section reports the descriptive and exploratory results produced by the validated V9 metric artifacts. Results are organized by research question and retain the analytical grain of each metric; no claim combines incompatible grains or treats self-reported evidence as behavioral telemetry.

A. RQ1: How do student developers utilize Generative AI, and how does this adoption shape their longitudinal perceptions of software engineering roles and project expectations?

RQ1 is supported by two distinct evidence streams: the M1 survey panel, which summarizes six perception families and separately reports self-reported AI use, and M2, which summarizes perceived disruption agreement for six software-engineering roles. All six cohort-by-semester/checkpoint cells in M1 have complete item coverage, with 46, 41, and 40 respondents in 2025.2 and 22, 20, and 18 respondents in 2026.1 at T_1 , T_2 , and T_3 , respectively. These are repeated cohort summaries, not an identified individual longitudinal panel.

The six perception families do not move as a single trajectory. These item means use the inherited 0–4 score scale, except for the separately reported autonomy/tool-balance item, which uses its observed 1–5 response scale. Values below are reported as mean $\pm SD$. In 2025.2, mean AI benefit increases from 2.13 ± 1.00 at T_1 to 2.25 ± 0.90 at T_3 , while five-year career impact decreases from 1.48 ± 1.01 to 1.33 ± 0.83 . Autonomy/tool balance changes from 3.09 ± 0.89 to 3.23 ± 0.95 . In 2026.1, AI benefit decreases from 2.00 ± 1.07 to 1.56 ± 0.98 , whereas autonomy/tool balance increases from 2.64 ± 0.85 to 3.44 ± 0.92 , and five-year career impact changes from 1.55 ± 0.91 to 1.67 ± 0.91 . The project-expectation families remain comparatively low: at T_3 , project challenges average 0.15 ± 0.36 in 2025.2 and 0.06 ± 0.24 in 2026.1, while project feeling averages 0.35 ± 0.77 and 0.50 ± 0.86 , respectively. These values describe distinct response families and should not be interpreted as one latent measure of AI dependence.

Figure 1 presents these six families with cohort-specific standard-deviation bars. Table V reports the corresponding endpoint values in compact form.

The raw usage fields provide a separate adoption description. The share reporting daily general AI use rises from 41.3% to 45.0% in 2025.2 and from 54.5% to 61.1% in 2026.1 between T_1 and T_3 . For software projects specifically, the daily-use share changes from 23.9% to 32.5% in 2025.2 and from 40.9% to 38.9% in 2026.1. The most frequently reported tasks include general research/questions, code generation, debugging, and documentation. For example, code generation rises from 65.2% to 77.5% in 2025.2, while documentation is reported by 72.7% of the 2026.1 T_1 respondents and by 66.7% at T_3 . These are self-reported frequencies and selected activities, not instrumented measures of tool use.

Reported tool use is concentrated in a small group of widely named tools but is not constant across cohorts. ChatGPT is reported by all 2025.2 respondents at T_1 and T_3 , and by 83.3% of the 2026.1 T_3 respondents. Gemini is reported by 77.5% of 2025.2 respondents at T_3 and 77.8% of 2026.1 respondents at T_3 . Prior AI-project experience also increases across the observed checkpoints, from 41.3% to 70.0% in 2025.2 and from 31.8% to 94.4% in 2026.1. Because the respondent counts and identities are not stable individual panels, these changes cannot be interpreted as within-person learning or adoption effects.

Career-impact topic coding is retained as an exploratory supplement rather than a validated result. Its largest categories are uncertainty or conditional statements and other/insufficient evidence; for example, together they account for 56.9% of topic assignments at 2025.2/ T_1 and 65.2% at 2026.1/ T_3 . The coding is rule-based and explicitly unvalidated, so these proportions are presented as exploratory descriptions only.

M2 shows a role-specific pattern rather than a uniform increase in perceived disruption. The values below are means on the 1–5 Likert agreement scale; each parenthetical value is the corresponding SD . In 2025.2, Frontend has the highest mean agreement at all three checkpoints: 2.74 ± 1.10 ,

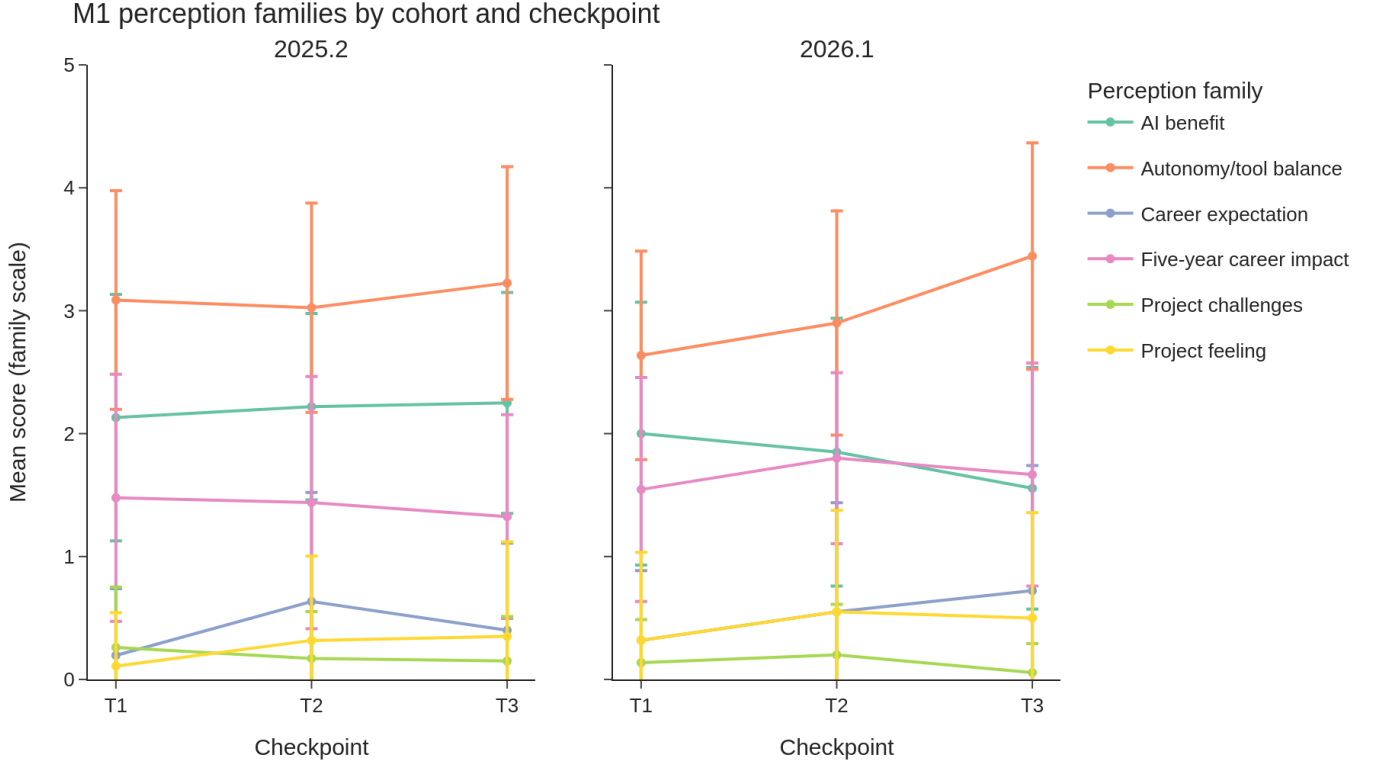


Fig. 1. M1 perception families by cohort and checkpoint. Points show means and bars show one standard deviation; family scales are retained from the underlying instruments.

TABLE V
M1 ENDPOINT SUMMARIES: MEAN $\pm$ SD

Family	2025.2 T_1	2025.2 T_3	2026.1 T_1	2026.1 T_3
AI benefit	2.13 ± 1.00	2.25 ± 0.90	2.00 ± 1.07	1.56 ± 0.98
Five-year career impact	1.48 ± 1.01	1.33 ± 0.83	1.55 ± 0.91	1.67 ± 0.91
Autonomy/tool balance	3.09 ± 0.89	3.23 ± 0.95	2.64 ± 0.85	3.44 ± 0.92
Project challenges	0.26 ± 0.49	0.15 ± 0.36	0.14 ± 0.35	0.06 ± 0.24
Project feeling	0.11 ± 0.43	0.35 ± 0.77	0.32 ± 0.72	0.50 ± 0.86

2.68 $\pm$ 1.08, and 2.73 $\pm$ 1.18. Project Manager remains among the lowest: 1.83 $\pm$ 1.02, 1.83 $\pm$ 0.86, and 2.08 $\pm$ 0.89. In 2026.1, Frontend is also high, moving from 3.14 $\pm$ 1.13 to 2.65 $\pm$ 1.09 and 3.28 $\pm$ 1.36; Scrum Master starts at 3.14 $\pm$ 1.25 at T_1 . Agreement is therefore role-specific and cohort-dependent; it does not support a general claim that all roles become progressively more exposed to replacement or transformation. Figure 2 provides the complete role-by-cohort/checkpoint view; its color scale is fixed to the 1–5 response range.

The privacy-preserving paired sensitivity analysis reinforces this caution. There are 38 paired respondents per role in 2025.2 and 16 per role in 2026.1. Changes are mixed rather than uniformly directional: in 2025.2, Project Manager shows the largest increase (+0.32) and Backend is nearly unchanged (−0.03); in 2026.1, QA increases by +0.69, while Scrum Master decreases by −0.31. These paired values are sensitivity summaries.

B. RQ2: How does the temporal density of repository activity contrast with the qualitative typification of human coordination friction across the project lifecycle?

RQ2 contrasts three descriptive evidence streams that operate at different grains. M3 measures repository activity timing, author participation, and concentration for 14 team-semesters. M4 measures clean source/test change at team-semester and checkpoint grain. M5 counts candidate coordination markers within a global transcript corpus by checkpoint. The measures are therefore reported side by side as complementary descriptions, not as a common-scale effect or a statistical correlation.

M3 shows a pronounced concentration of repository activity around the final evaluator anchor. In the semester-pooled rolling view, total commits increase from 95 at 21 days before the anchor to 159 one day before, peak at 302 one day after, and fall to 28 at seven days after. At the anchor itself, all 14 team-semesters have observed activity; at −7 days, only 5 of 14 are active, and at +7 days, 7 of 14 remain active.

M2 perceived role disruption agreement

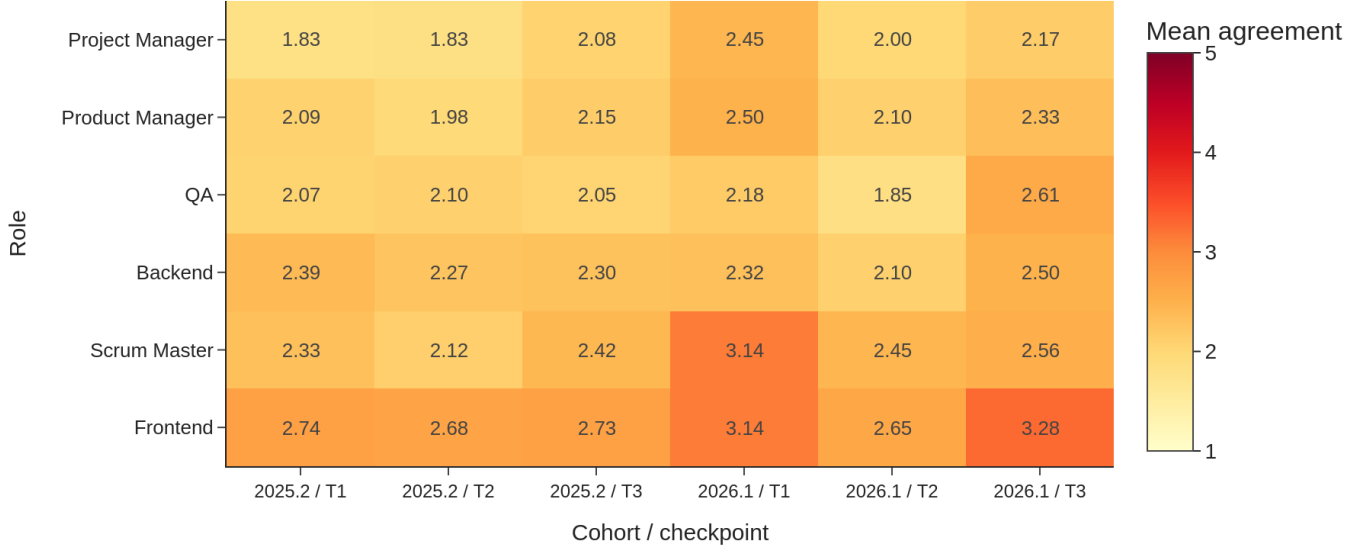


Fig. 2. M2 perceived role-disruption agreement by cohort and checkpoint. Cells show mean agreement on the 1–5 Likert scale.

The corresponding median commit count per team-semester is 12.5 at the anchor and 0.5 at +7 days. These rolling windows overlap, so adjacent points are temporal summaries rather than independent observations. Figure 3 visualizes this pooled temporal shape; the dashed line marks the final evaluator anchor.

The checkpoint participation view shows the same concentration with substantial heterogeneity. Across the 14 team-semesters, 7 have any post-checkpoint activity. The final-window share of all commits ranges from 9.5% to 100.0%; three team-semesters have fewer than ten total commits, and their shares should be treated as small-denominator descriptions. In the author concentration output, the median pre-anchor window contains 12.5 commits, 3.5 active days, and 3 authors, with a median maximum-author share of 0.50 and median Gini coefficient of 0.22. Post-anchor windows are frequently inactive; where activity exists, concentration statistics are not interpreted for no-activity cases.

M4 reproduces the temporal increase using a narrower clean-change definition. Pooled clean churn rises from 21,718 lines across 7 observed team-semesters at T_1 , to 47,545 lines across 10 at T_2 , and 136,325 lines across all 14 at T_3 . The median clean churn per team-semester rises from 66.5 lines at T_1 to 3,232.5 at T_2 and 6,975.5 at T_3 . Clean change accounts for 42.9%, 19.6%, and 1.0% of all recorded churn at the three checkpoints, respectively, because excluded or non-measurement paths dominate the raw line-change total in the final checkpoint. This distinction is why M4 is not interchangeable with total repository activity.

M5 provides a separate textual axis for the 2025.2 transcript corpus. The corpus contains 35, 37, and 28 transcript chunks at T_1 , T_2 , and T_3 , respectively, with two source sessions at each

checkpoint. Candidate marker density increases from 0.515 to 1.132 and 1.287 markers per 1,000 transcript tokens. Marker counts are 12, 54, and 40, over 23,296, 47,705, and 31,087 tokens. The composition also changes: integration markers account for 20 of 54 markers at T_2 , while blockers account for 7 of 40 at T_3 . These are deterministic lexical matches in a global corpus, not diagnoses of coordination failure; transcript coverage is unavailable for 2026.1.

Table VI consolidates the principal RQ2 values while retaining their distinct units and analytical grains. Figure 4 shows the M4 and M5 checkpoint trajectories in separate panels.

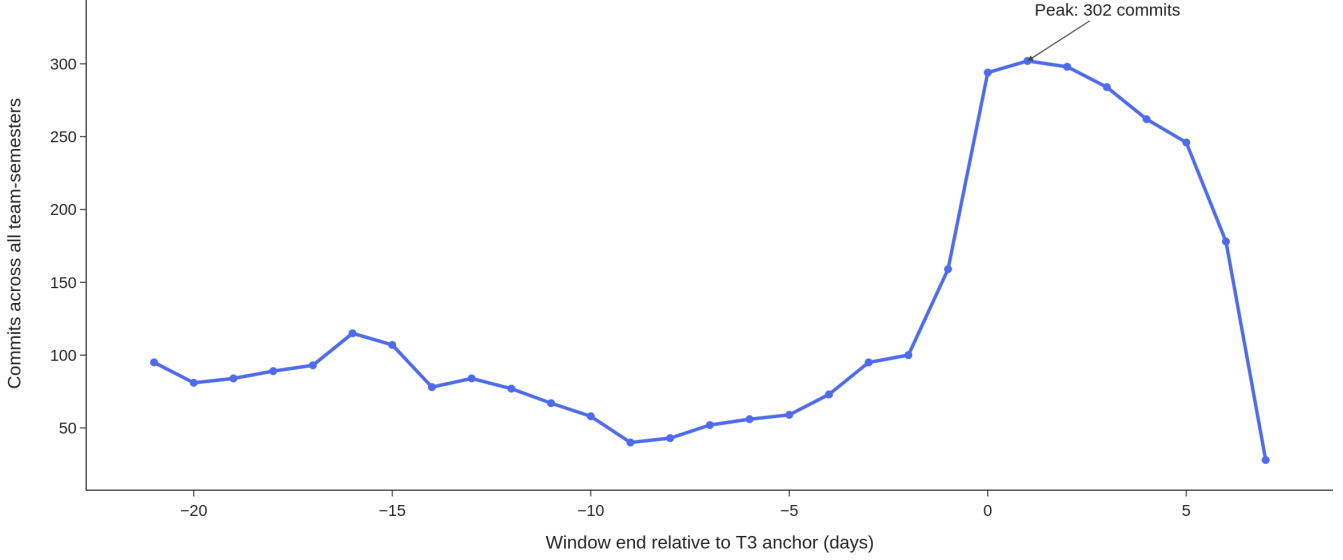
Taken together, M3 and M4 show that repository-visible activity and clean source/test change become most concentrated near the final checkpoint, while M5 shows an increase in candidate coordination-language density across the observed 2025.2 transcript corpus. The three patterns are compatible with the RQ2 descriptive contrast within the observed evidence streams.

C. RQ3: How does the quality of upfront architectural artifacts (evaluated at baseline T_1) associate with late-stage destructive rework and independent, blind-scored project outcomes?

RQ3 is evaluated at team-semester grain across 14 observations. The analysis separates observable upfront architectural artifacts from their semantic interpretation, separates rework magnitude from a baseline-conditioned ratio, and treats evaluator outcomes as independent T_3 scorecard summaries. All associations are reported as descriptive and exploratory.

M6a (T_1 architectural-artifact evidence) records whether a T_1 architectural artifact was observed and its repository-visible structural scope. An artifact is present in 4 of 9 team-semesters

M3 pooled rolling repository activity

Fig. 3. M3 pooled rolling repository activity. The vertical marker identifies the T_3 evaluator anchor; the annotation identifies the pooled peak.TABLE VI
RQ2 TEMPORAL SUMMARY WITH EXPLICIT UNITS

Metric	Unit / grain	T_1 or -7	T_2 or -1	T_3 or 0	Post-anchor	Coverage
M3 activity	Pooled commits / rolling window	52 (-7)	159 (-1)	294 (0)	302 (+1); 28 (+7)	14 team-semesters
M4 clean change	Clean churn lines / checkpoint	21,718	47,545	136,325	–	7 / 10 / 14 observed
M5 lexical evidence	Markers per 1,000 tokens / global corpus	0.515	1.132	1.287	–	2025.2 only

in 2025.2 and in all 5 team-semesters in 2026.1. Among the 2025.2 observations, the median T_1 file count is zero and the median T_1 line delta is zero; among the 2026.1 observations, the corresponding medians are 4 files and 114 changed lines. The largest observed T_1 line delta is 232 in 2025.2 and 222 in 2026.1. These are structural observations, not measures of architectural quality or planning intent. Figure 5 summarizes the cohort-level presence pattern.

M6b (structured T_1 planning-content evidence) adds structured evidence from T_1 commit subjects for 9 of the 14 team-semesters; 5 cases have no T_1 subjects and were not measured. The reviewed fields cover planning evidence, goals, architecture/design, task decomposition, risk/dependency, and literal evidence quotes. They are retained as separate exploratory fields, with no composite planning or architectural-quality score.

M7 shows that repository inactivity is not equivalent to architectural-artifact absence. At T_1 , 7 of 9 team-semesters are inactive in the 2025.2 seven-day window and 3 of 5 are inactive in 2026.1; by T_3 , no team-semester is inactive in either cohort. M7 is therefore used as a repository timing and coverage diagnostic, not as a planning-omission measure and not as an M9 predictor.

M8 reports late-stage changes to previously observed clean paths as a file-provenance proxy for destructive rework.

Among the 14 team-semesters, 12 are baseline-eligible. The T_3 clean-rework ratio ranges from 0.000 to 1.000 among the reported observations, while clean rework magnitude ranges from zero to 15,518 lines. For example, the largest magnitude occurs in 2025.2/TEAM_09, whereas 2025.2/TEAM_08 and 2026.1/TEAM_03 have a ratio of 1.000 but very different absolute magnitudes (172 and 196 lines). Baseline-ineligible cases remain separate and are not interpreted as stable or low-rework cases. Because path provenance does not establish semantic defects, M8 is not a direct defect measure. Figure 6 displays the artifact-scope/rework profile on a logarithmic rework axis, preserving baseline eligibility.

The primary M9 associations do not support a general monotonic relationship between T_1 artifact scope and later rework. Across all 14 team-semesters, Spearman’s ρ is 0.206 for T_1 file count versus T_3 clean-rework magnitude and 0.301 for log-transformed T_1 line scope versus that magnitude. Within the 12 baseline-eligible cases, the corresponding associations with the T_3 clean-rework ratio are -0.032 and -0.060 . Their exploratory p-values range from 0.296 to 0.922, and the small sample does not support confirmatory inference.

The independent blind-scored T_3 outcomes are similarly weakly associated with the structural predictors. For T_1 file count, the correlations range from -0.079 for engagement/participation to 0.271 for technical complexity; for log-

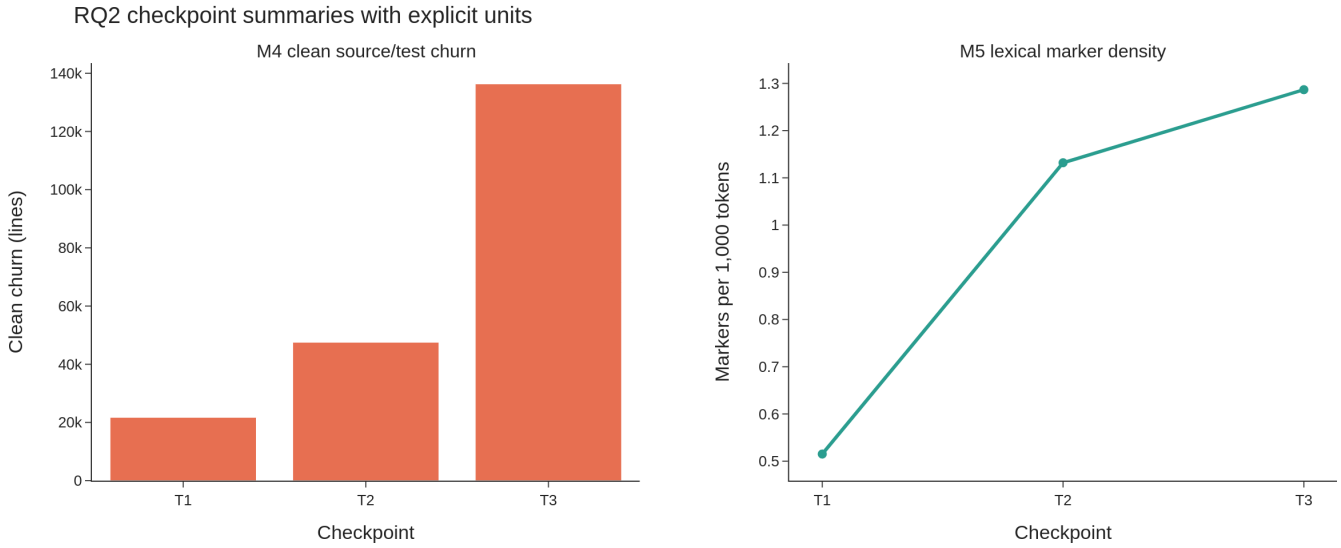


Fig. 4. RQ2 checkpoint summaries with explicit units. M4 clean source/test churn and M5 lexical marker density use separate panels and scales.

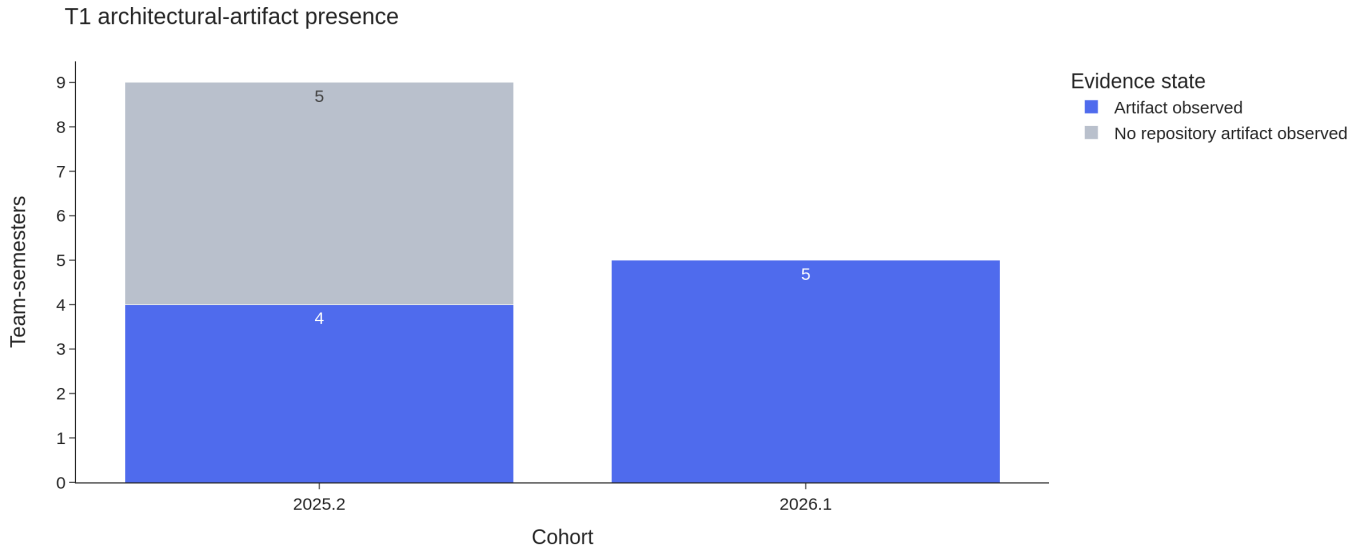


Fig. 5. T1 architectural-artifact presence by cohort. “No repository artifact observed” is an observation state, not evidence that planning did not occur elsewhere.

transformed T1 line scope, they range from -0.143 to 0.222 across the four outcomes. The outcome summaries themselves vary across the 14 team-semester: mean project progress is 3.55 ± 0.67 , while the other three recorded outcome means range from 1.76 ± 0.24 to 1.88 ± 0.25 on their respective scorecard scales.

The approved M6b structured planning-content fields produce a separate, smaller exploratory view with $n = 9$. Its largest absolute association with clean-rework magnitude is $\rho = -0.408$ for task-decomposition evidence; its largest positive association with an evaluator outcome is $\rho = 0.479$ for task-decomposition evidence versus scope applicability. Leave-one-out diagnostics for the M6a rework analyses pro-

duce ranges of 0.078 – 0.377 for T1 file count and 0.194 – 0.494 for T1 line scope when predicting rework magnitude. For the eligible-stratum rework ratio, the corresponding ranges are -0.222 – 0.165 and -0.221 – 0.137 . These diagnostics expose sensitivity to individual team-semester rather than confidence intervals. Table VII reports the primary M9 association values; Figure 7 visualizes the leave-one-out ranges.

Overall, the RQ3 evidence establishes a measurable contrast between observable T1 artifact scope, late clean-path changes, and independent evaluator outcomes, but it does not establish that artifact quality causes less destructive rework or better project outcomes. The principal result is therefore a bounded, exploratory association profile with explicit missingness and

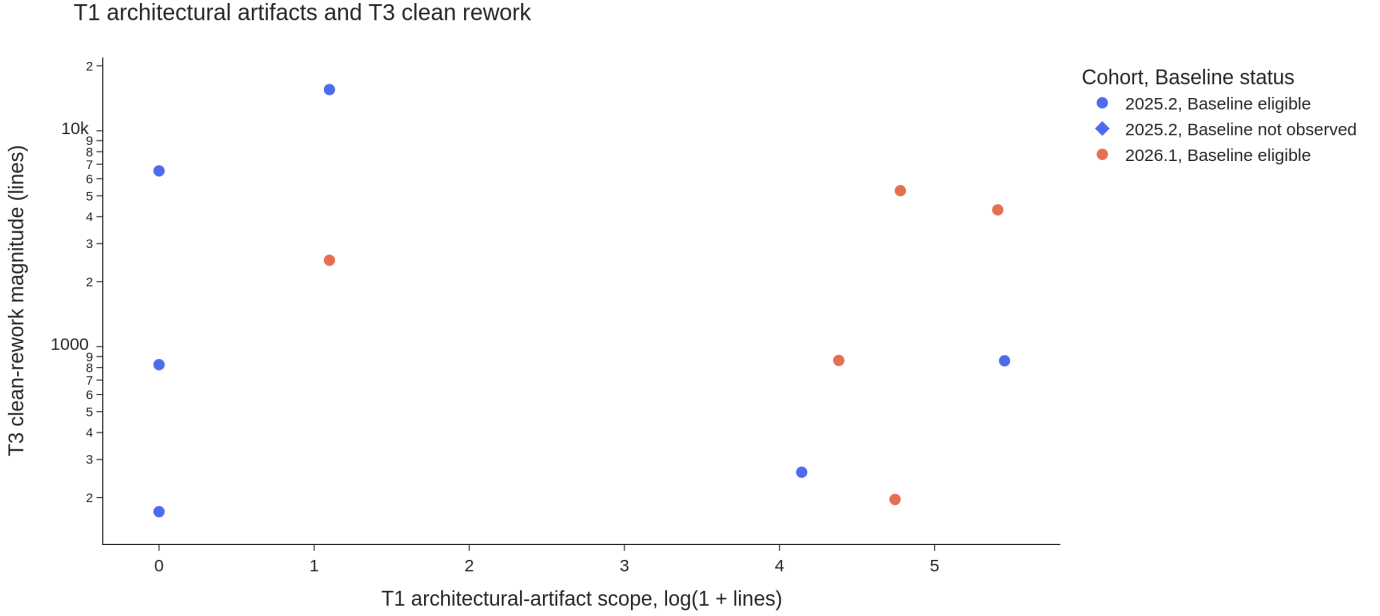


Fig. 6. T1 architectural-artifact scope versus T3 clean-rework magnitude. Points represent team-semesters; the vertical scale is logarithmic. Clean rework is a file-provenance proxy, not semantic defect validation.

TABLE VII
PRIMARY RQ3 ASSOCIATION RESULTS

Predictor	Outcome	Stratum	<i>n</i>	ρ	Exploratory <i>p</i>
T1 artifact file count	T3 rework magnitude	All team-semesters	14	0.206	0.479
T1 artifact line scope	T3 rework magnitude	All team-semesters	14	0.301	0.296
T1 artifact file count	T3 rework ratio	Baseline-eligible	12	-0.032	0.922
T1 artifact line scope	T3 rework ratio	Baseline-eligible	12	-0.060	0.853
T1 artifact file count	Project progress	All team-semesters	14	0.100	0.733
T1 artifact file count	Scope applicability	All team-semesters	14	0.025	0.931
T1 artifact file count	Technical complexity	All team-semesters	14	0.271	0.349
T1 artifact file count	Engagement/participation	All team-semesters	14	-0.079	0.790
T1 artifact line scope	Project progress	All team-semesters	14	0.096	0.745
T1 artifact line scope	Scope applicability	All team-semesters	14	0.038	0.897
T1 artifact line scope	Technical complexity	All team-semesters	14	0.222	0.446
T1 artifact line scope	Engagement/participation	All team-semesters	14	-0.143	0.626

influence diagnostics, not a causal planning-effect estimate.

V. DISCUSSION

The results support a qualified answer to the three research questions. Student developers report frequent use of Generative AI across research, code generation, debugging, documentation, and related activities, but the perception families do not form a single longitudinal trajectory. Repository activity and clean source/test change concentrate near the final evaluation anchor, while candidate coordination-language density increases in the observed transcript corpus. Finally, the study finds observable variation in upfront architectural-artifact scope, but no robust monotonic association with late clean-path rework or independent blind-scored outcomes. The central implication is therefore not that a larger artifact necessarily produces a better project, but that AI-assisted development makes the quality and visibility of shared intent

consequential while leaving its effects difficult to identify with small observational samples.

A. From local acceleration to system-level coordination

RQ1 and RQ2 describe two sides of AI-assisted development. Survey responses show that students use Generative AI for concrete development activities, while M3 and M4 show that repository-visible activity intensifies around final integration. M5 adds textual evidence that candidate coordination markers are more dense at later checkpoints in the observed 2025.2 corpus. These results do not establish that AI use caused coordination friction; the relevant measurement boundaries are consolidated in Threats to Validity. They do, however, weaken a simple productivity story in which faster local code production automatically yields smoother system integration.

The more defensible interpretation is a shift in the bottleneck. Generative AI can reduce the effort required to produce

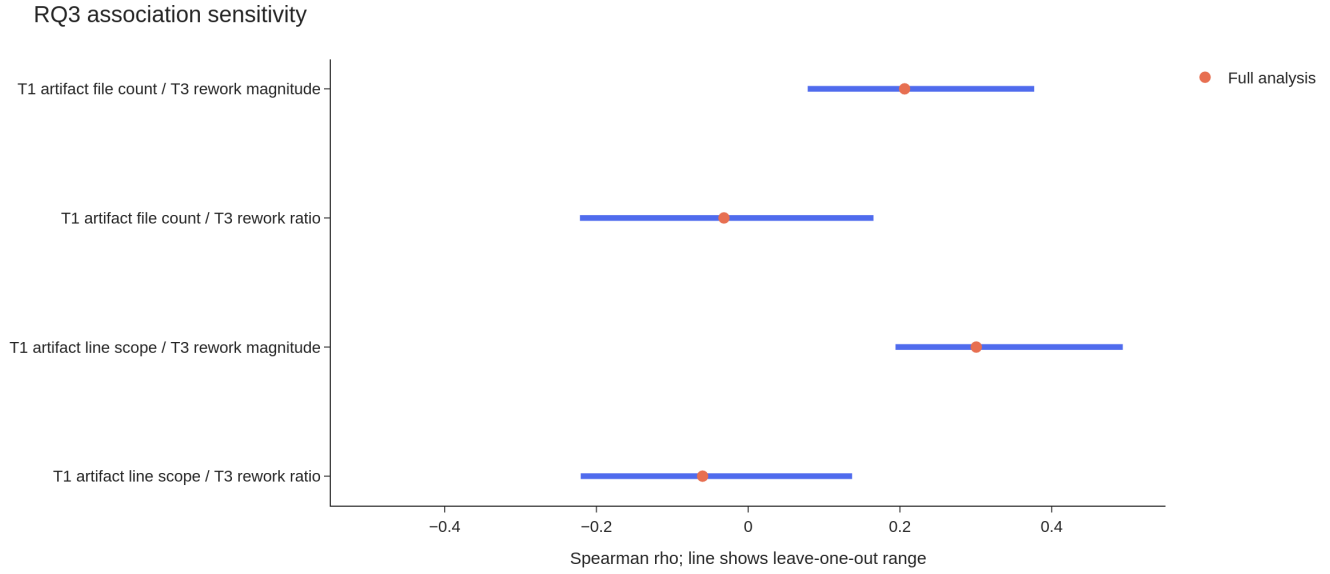


Fig. 7. RQ3 association sensitivity. Points show full-analysis Spearman values; horizontal segments show leave-one-out ranges. These ranges are descriptive diagnostics, not confidence intervals.

local implementation fragments, but it does not remove the need to agree on architecture, interfaces, task boundaries, dependencies, or acceptance conditions. In a multi-person project, those agreements remain coordination objects even when code is produced more quickly. The late concentration observed in M3/M4 is consistent with a process in which integration pressure accumulates until the system must be assembled under deadline, while M5 suggests that coordination-related language is already present in the shared record. This is a process interpretation, not a causal estimate.

B. Architectural artifacts: necessary evidence, not a quality score

RQ3 provides an important qualification to strong planning narratives. M6a measures whether an architectural artifact was observed in the repository and how much structural scope it occupied; M6b extracts literal, structured planning-content evidence from T1 commit subjects. Neither metric directly measures architectural quality. Repository absence cannot establish that a team did not plan elsewhere, and a larger artifact cannot establish that its design was coherent or useful.

This distinction explains why the M9 associations are modest and unstable rather than uniformly directional. T1 artifact scope has small positive associations with T3 rework magnitude in the complete team-semester set, while the baseline-eligible rework-ratio associations are near zero and slightly negative. Associations with blind-scored outcomes are likewise mixed and small. The leave-one-out ranges show that individual team-semesters can materially change the observed coefficient. The result is not evidence against planning; it is evidence against treating repository-visible artifact volume as a validated measure of planning quality.

The distinction also matters for interpretation of the tail cases. A team with no observed T1 artifact and a team with an artifact that is incomplete, misaligned, or unused are not equivalent cases. Conversely, a high rework ratio does not by itself identify a defect: M8 follows clean-path provenance and cannot determine whether a change was semantically destructive. The empirical chain is consequently useful as an auditable observation system, but not as a single planning-quality scale.

C. Why Spec-Driven Development is a plausible response

The results provide a strong rationale for discussing Spec-Driven Development (SDD). The argument is not that this study tested SDD, nor that SDD is proven to reduce rework. Rather, the observed combination of rapid local generation, late integration concentration, repeated architectural reformulation in the project records, and persistent coordination demands identifies precisely the class of problem that SDD is designed to address: keeping intent, constraints, architecture, and tasks explicit before and during implementation.

Current SDD guidance describes a workflow in which a specification is carried through technical planning, task decomposition, implementation, and explicit convergence checks [1], [2]. This is aligned with the mechanism suggested by the present evidence. If Generative AI makes implementation cheap while leaving system-level decisions underspecified, a process that externalizes those decisions can reduce ambiguity before it is multiplied across generated code and parallel contributors. The potential benefit is not documentation for its own sake; it is a shared, inspectable context for coordinating people and AI tools.

The claim must remain conditional. The current data do not compare SDD with another process, do not randomize teams

or interventions, and do not measure specification adoption as a treatment. SDD is therefore a theoretically grounded intervention proposal for future evaluation, not a causal conclusion of this study. A suitable follow-up would compare teams using an explicit specification-to-plan-to-task workflow with teams using their usual process, while measuring architectural change, integration timing, rework provenance, coordination evidence, and independently scored outcomes.

D. Implications for Agile practice

These findings do not imply that iterative development should be abandoned. They suggest that Agile iteration under Generative AI requires a clearer object of iteration. When code can be generated rapidly, iteration should include shared architectural decisions, executable acceptance conditions, interface contracts, and explicit dependency boundaries, rather than treating working code as the only visible progress signal. This interpretation is compatible with Agile values while challenging a purely code-first implementation rhythm.

The practical implication is a stronger planning layer, not a return to a fixed plan that cannot change. Specifications should remain revisable, but their revisions should be explicit, reviewable, and connected to the tasks and implementation they govern. That is the specific space in which SDD deserves to be tested: as a way to preserve adaptability while making the intent behind AI-assisted change inspectable.

E. Scope of the interpretation

The Discussion is intentionally bounded by the evidence design. RQ1 uses repeated cohort summaries rather than a stable individual panel; the remaining coverage, window, proxy, sample, and exploratory-association boundaries are consolidated in Threats to Validity. These limits make the findings useful for mechanism-oriented hypothesis generation, but not for estimating the causal effect of Generative AI, architectural planning, or SDD on project outcomes. The strongest conclusion is narrower: under AI-assisted development, local generation speed does not eliminate the need for explicit system-level coordination, and SDD is a plausible, testable response to that coordination problem.

VI. THREATS TO VALIDITY

The constructs are measured through complementary but imperfect proxies. M1 and M2 are self-reported survey measures rather than behavioral telemetry. M6a captures repository-visible T1 architectural artifacts, while M6b extracts structured evidence from commit subjects. M6b remains exploratory and its categories were subject to human review; neither measure establishes architectural quality or planning intent. M8 uses file-path provenance to identify changes to previously observed clean paths, not semantic defects or destructive intent. M5 is based on deterministic lexical markers in transcript text; its counts are candidate textual evidence, not direct diagnoses of coordination friction. These distinctions limit the interpretation of every metric and prevent their

being treated as interchangeable measures of quality, effort, or productivity.

The observational design and small sample constrain causal and external validity. M9 associations are exploratory, not causal or confirmatory, and leave-one-out ranges show sensitivity to individual team-semesters. Analytical grains differ across student responses, team-semesters, rolling windows, and a global transcript corpus, so denominators cannot be pooled implicitly. Rolling windows overlap, so adjacent values are not independent observations. The study covers 14 team-semesters in one educational setting; transcript evidence is available only for 2025.2, and Git cannot observe planning or collaboration outside the repositories. Five M6b cases lacked T1 commit subjects and were not measured; missing evidence was not converted into a low score. The external SDD literature therefore motivates a testable intervention hypothesis, but this study neither implements nor evaluates SDD. Its results support the need to test more explicit specification and coordination practices under Generative AI, not a claim that SDD has already improved outcomes.

VII. CONCLUSION

Do we still need to be Agile? Yes, but Generative AI changes what effective agility requires. Iteration, feedback, and adaptation remain valuable; however, faster local implementation does not remove the need for shared architectural intent, explicit interfaces, dependency boundaries, and reviewable acceptance conditions. Across the three research questions, this study shows that self-reported AI use is widespread and task-oriented, repository activity concentrates near final integration, and observable upfront architectural artifacts do not translate into a simple monotonic pattern of rework or project outcomes.

The study's contribution is to make this tension empirically inspectable through complementary evidence: longitudinal perception and adoption reports, temporal repository measures, transcript-based coordination evidence, structural T1 artifacts, path-based rework measures, and independent evaluator scorecards. Together, these results suggest that GenAI-assisted development shifts a larger share of engineering difficulty away from producing local code and toward coordinating system-level intent. The practical implication is not a return to immutable Big Design Up Front, but a stronger planning layer that remains iterative, explicit, and shared by people and AI tools.

Spec-Driven Development is a plausible way to instantiate that planning layer: requirements become specifications, specifications become technical plans and tasks, and implementation is repeatedly checked against those artifacts. This study does not evaluate SDD as an intervention, so its value here is a grounded and testable proposition rather than an established causal conclusion. Future work should compare ordinary practice with an iterative SDD condition while measuring specification quality, consented AI-tool use, integration timing, coordination evidence, rework provenance, and independently scored outcomes.

REFERENCES

- [1] GitHub Blog, “Spec-driven development with ai: Get started with a new open source toolkit,” September 2025. [Online]. Available: <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- [2] Thoughtworks, “Spec-driven development,” 2025. [Online]. Available: <https://www.thoughtworks.com/en-br/insights/decoder/s/spec-driven-development>
- [3] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, “Manifesto for agile software development,” 2001. [Online]. Available: <https://agilemanifesto.org/>
- [4] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.
- [5] T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*. Dorset House, 1987.
- [6] A. Carleton, D. Falessi, H. Zhang, and X. Xia, “Generative ai: Redefining the future of software engineering,” *IEEE Software*, vol. 41, no. 6, pp. 34–37, 2024.
- [7] L. Banh, F. Holldack, and G. Strobel, “Copiloting the future: How generative ai transforms software engineering,” *Information and Software Technology*, vol. 183, p. 107751, 2025.
- [8] D. Russo *et al.*, “Navigating the complexity of generative ai adoption in software engineering,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, 2024.
- [9] S. Barke, M. B. James, and N. Polikarpova, “Grounded copilot: How programmers interact with code-generating models,” *Proceedings of the ACM on Programming Languages*, vol. 7, 2023.
- [10] M. Yabaku and S. Ouhbi, “On usage and assessment of generative ai by computer science students in software development projects,” in *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*, 2024.
- [11] R. Verdecchia, “Architectural technical debt identification: Moving forward,” in *2018 IEEE International Conference on Software Architecture Companion (ICSA-C)*, 2018, pp. 43–44.
- [12] M. Nayeibi, Y. Cai, R. Kazman, G. Ruhe, Q. Feng, C. Carlson, and F. Chew, “A longitudinal study of identifying and paying down architecture debt,” in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 171–180.
- [13] W. Cunningham, “The wycash portfolio management system,” in *OOP-SLA ’92 Experience Report*. ACM, 1992.
- [14] M. Fowler, “Technical debt quadrant,” 2009. [Online]. Available: <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>
- [15] P. Kruchten, R. L. Nord, and I. Ozkaya, “Technical debt: From metaphor to theory and practice,” *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.
- [16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [17] C. Spearman, “The proof and measurement of association between two things,” *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.
- [18] A. Nguyen-Duc *et al.*, “A systematic literature review on generative ai in software engineering,” *Preprints*, May 2026. [Online]. Available: <https://www.preprints.org/manuscript/202605.1638>
- [19] IEEE Computer Society, “Transforming software engineering processes through generative ai,” *IEEE Computer*, July 2025. [Online]. Available: <https://www.computer.org/csdl/magazine/co/2025/07/11052608/27RKcQkdNg4>
- [20] Cambridge Advance Online, “The contextual gap in genai for software engineering,” July 2026. [Online]. Available: <https://advanceonline.cam.ac.uk/blog/the-contextual-gap-in-genai-for-software-engineering>
- [21] Codebridge, “The hidden costs of ai-generated software: Why ‘it works’ isn’t enough,” February 2026. [Online]. Available: <https://www.codebridge.tech/articles/the-hidden-costs-of-ai-generated-software-why-it-works-isnt-enough>
- [22] M. Staron and S. Abrahão, “Exploring generative ai in automated software engineering,” *IEEE Software*, vol. 42, no. 3, pp. 142–145, 2025.
- [23] K.-J. Stol and B. Fitzgerald, “The abc of software engineering research,” *ACM Transactions on Software Engineering and Methodology*, vol. 27, no. 3, 2018.
- [24] P. Runeson, M. Host, A. J. Rainer, and B. Regnell, “Case study research in software engineering: Guidelines and examples,” *Empirical Software Engineering*, vol. 14, 2009.
- [25] Z. Li, P. Avgeriou, and P. Liang, “A systematic mapping study on technical debt and its management,” *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.
- [26] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Boston, B. McGrew, S. Karthik, and *et al.*, “Evaluating large language models trained on code,” in *arXiv preprint arXiv:2107.03374*, 2021.
- [27] C. Williams, M. Jones, and R. Patel, “Generative ai and software engineering education: Opportunities, risks, and design constraints,” in *Proceedings of the International Conference on Software Engineering Education and Training*, 2024.
- [28] S. Miller, J. Ferris, and A. Chen, “Coordination signals and human-ai teaming in software engineering courses,” in *Proceedings of the ACM/IEEE International Conference on Software Engineering*, 2023.
- [29] J. Pérez, A. Moreno, and E. Lauri, “Planning quality, rework, and team coordination in project-based software courses,” *IEEE Transactions on Education*, vol. 65, no. 2, pp. 214–223, 2022.